

Migracija Clipper aplikacije u Firebird

🌟 claude.ai/chat/bedbddde-b393-4500-ab26-f480776f60bf



BY ANTHROPIC

Use the up and down arrow keys to move between messages.

Imam staru aplikaciju napisanu u Clipper 5.2 koja kao bazu podataka koristi .DBF datoteke. U njoj imam napravljeno nekoliko stotina računa. Te račune želim importirati u Firebird 3.0 bazu podataka. Je li to moguće napraviti u Pythonu?

Jul 8

Artikuliranje rješenja za migraciju DBF datoteka

Artikuliranje rješenja za migraciju DBF datoteka

Da, to je sasvim izvedivo u Pythonu. To je česta migracija (Clipper/dBase → moderna SQL baza) i postoji dobra podrška kroz gotove biblioteke. Evo kako bih pristupio tome:

1. Čitanje DBF datoteka

Najbolja opcija je **dbfread** (za čisto čitanje) ili **simple-dbf** / **ydbf**. Ako imate i memo polja (.dbt ili .fpt datoteke uz .dbf), dbfread ih automatski povezuje.

bash

```
pip install dbfread
```

Bitno za Clipper baze: gotovo sigurno koristite kodnu stranicu **CP852** (Latin 2, srednja Europa) zbog hrvatskih dijakritičkih znakova (č, ć, š, ž, đ). Ako ne postavite ispravan encoding, ti će se znakovi krivo pročitati.

python

```
from dbfread import DBF
```

```
table = DBF('RACUNI.DBF', encoding='cp852', ignore_missing_memofile=True)
```

```
for record in table:
    print(record) # dict s nazivima polja i vrijednostima
```

Ako dbfread ne pogodi encoding automatski ili vam vraća čudne znakove, probajte i cp850 ili čak windows-1250 — ovisi kako je aplikacija zapisivala podatke.

2. Spajanje na Firebird 3.0

Preporučam službeni driver **firebird-driver** (noviji, aktivno se održava) umjesto starijeg fdb:

bash

```
pip install firebird-driver
```

python

```
import firebird.driver as fb
```

```
con = fb.connect(
    database='localhost/3050:/putanja/do/baze.fdb',
    user='SYSDBA',
    password='masterkey'
)
cur = con.cursor()
```

3. Skica cijelog procesa

python

```
from dbfread import DBF
import firebird.driver as fb
```

```
# Čitanje DBF-a
table = DBF('RACUNI.DBF', encoding='cp852', ignore_missing_memofile=True)
```

```
# Spajanje na Firebird
con = fb.connect(database='localhost/3050:/data/baza.fdb',
                  user='SYSDBA', password='masterkey')
cur = con.cursor()
```

```

insert_sql = """
    INSERT INTO RACUNI (BROJ_RACUNA, DATUM, KUPAC, IZNOS)
    VALUES (?, ?, ?, ?)
"""

batch = []
for record in table:
    if record.get('DELETED'): # dbfread obično već filtrira obrisane, ali
    provjerite
        continue
    batch.append((
        record['BROJRAC'],
        record['DATUM'],
        record['KUPAC'],
        record['IZNOS']
    ))

    if len(batch) >= 500:
        cur.executemany(insert_sql, batch)
        con.commit()
        batch.clear()

if batch:
    cur.executemany(insert_sql, batch)
    con.commit()

con.close()

```

Na što obratiti pažnju

- **Obrisani zapisi** – Clipper označava obrisane retke posebnim bajtom (*) na početku sloga, ali ih fizički ne briše dok se ne pokrene PACK. dbfread po defaultu preskače te zapise, što je obično poželjno.
- **Nazivi polja u DBF-u** – ograničeni na 10 znakova (Clipper/dBase III/IV), pa provjerite mapiranje na nazive stupaca u Firebirdu.
- **Datumski format** – DBF datumska polja (D) dbfread automatski pretvara u Python date objekte, što se lijepo poklapa s Firebird DATE tipom.
- **Numerička polja** – DBF N tip može imati decimalne zapetice zapisane kao tekst; dbfread ih vraća kao Decimal ili int, provjerite da odgovara tipu stupca u Firebirdu (NUMERIC(x,y)).
- **Indeksi (.ntx/.ndx/.cdx)** – ne trebaju vam za migraciju, sadrže samo indeksne strukture, ne podatke.
- **Vežane tablice** – ako imate glavu i stavke računa u dvije DBF datoteke povezane preko šifre, migrirajte prvo glavu pa stavke, uz očuvanje FK veze.

Ako mi kažete strukturu polja u DBF-u (nazive i tipove) i ciljanu shemu tablice u Firebirdu, mogu vam pripremiti konkretniju skriptu za mapiranje i uvoz.

Ne želim skriptu, želim da zajedno napravimo aplikaciju koja će to raditi.

Jul 8

Razjasnio potrebne detalje pre nego što počne razvoj

Razjasnio potrebne detalje pre nego što počne razvoj

Odlično, idemo to napraviti kao pravu aplikaciju. Prije nego počnem pisati kod, trebam razjasniti nekoliko ključnih stvari koje će odrediti arhitekturu:

Q: Kakvu vrstu aplikacije želite - s grafičkim sučeljem (GUI) ili je dovoljna konzolna aplikacija s jasnim ispisom napretka? A: Grafičko sučelje (GUI)

Q: Postoji li već tablica u Firebird bazi za račune, ili je treba tek kreirati? A: Tablica već postoji

Q: Hoće li se ovo koristiti jednom (jednokratni import) ili očekujete da ćete aplikaciju koristiti više puta (npr. redovita sinkronizacija)? A: Više puta / redovito

Jul 8

Osmislio generičku aplikaciju s mapiranjem polja i trajnom konfiguracijom

Osmislio generičku aplikaciju s mapiranjem polja i trajnom konfiguracijom

Super. S obzirom da se radi o redovitoj upotrebi, aplikacija treba imati:

- **Spremljene postavke konekcije** (putanja do DBF-a, podaci za Firebird) da ih ne upisujete svaki put
- **Praćenje već uvezenih zapisa** (da se isti računi ne uvezu dvaput)
- **Pregled podataka prije uvoza** i log/izvještaj nakon uvoza
- **Mapiranje polja** DBF → Firebird tablica

Da bih napravio stvarno mapiranje polja (a ne izmišljao nazive), trebam vidjeti pravu strukturu. Najbrži put:

1. **Uploadajte mi DBF datoteku** (ili barem jednu manju/testnu ako je osjetljivo) — mogu pročitati strukturu polja izravno

2. Recite mi ili zalijepite **strukturu Firebird tablice** (npr. izlaz `SELECT * FROM RDB$RELATION_FIELDS WHERE RDB$RELATION_NAME='RACUNI'` ili samo popis stupaca i tipova, ili DDL kojim je tablica kreirana)

Dodatno, da definiram GUI:

Q: Koji GUI okvir preferirate za aplikaciju? A: Tkinter (jednostavno, bez dodatnih instalacija)

Q: Kako želite prepoznati koji su računi već uvezeni (da se ne dupliciraju)? A: Po broju računa (jedinstven identifikator)

Jul 8

Odlično, imamo plan: **Tkinter** GUI, dedup po broju računa. Sad mi treba stvarna struktura podataka da mapiranje bude točno, a ne izmišljeno.

Možete li:

1. **Uploadati DBF datoteku** (ili njenu manju/testnu kopiju) u ovaj razgovor — pročitat ću strukturu polja direktno
2. Poslati **strukturu ciljne Firebird tablice** — najlakše kao DDL (`SHOW TABLE RACUNI` u isql-u, ili iz Firebird alata poput FlameRobin/DBBeaver izvezite DDL), ili jednostavno popis stupaca s tipovima

Ako imate i memo/veznu tablicu za stavke računa (glava + stavke), uploadajte oboje.

Dok čekam to, mogu vam reći kako će izgledati struktura aplikacije koju gradimo:

```
racun_import/
├─ main.py                # pokretanje GUI-ja
├─ config.py              # spremanje/učitavanje postavki (JSON)
├─ dbf_reader.py          # čitanje DBF-a preko dbfread
├─ firebird_client.py     # konekcija i upis u Firebird
├─ mapping.py             # mapiranje polja DBF -> Firebird
├─ import_log.py          # praćenje uvezenih brojeva računa (lokalna SQLite
ili tablica u Firebirdu)
├─ gui/
│   ├─ main_window.py    # glavni prozor s tabovima
│   ├─ settings_tab.py   # postavke konekcije
│   ├─ preview_tab.py    # pregled podataka prije uvoza
│   └─ import_tab.py     # pokretanje uvoza + log
└─ requirements.txt
```

Za praćenje uvezenih računa predlažem dodati mali kontrolni stupac ili tablicu u samoj Firebird bazi (npr. tablica `IMPORT_LOG` s brojem računa i datumom uvoza), umjesto lokalne datoteke — tako je robusnije ako aplikaciju pokrećete s više računala.

Javite mi strukturu podataka pa krećemo pisati kod.

-- VPRZG definition

```
CREATE TABLE VPRZG ( IDVPRZG BIGINT NOT NULL, IDFIRME BIGINT
NOT NULL, VK SMALLINT, BRDOK BIGINT NOT NULL, IDTVRTKE BIGINT
NOT NULL, SIFRATVRTKEPJ VARCHAR(6), DATDOK DATE NOT NULL,
DATDANA INTEGER, DATVAL DATE, DATISP DATE, IDPOSJED BIGINT
NOT NULL, SIFTROSKA VARCHAR(20), IDSKLAD BIGINT NOT NULL,
IDROBACJENIK BIGINT NOT NULL, IDTIPOTPREME INTEGER,
ZASTAMPU CHAR(1) DEFAULT 'F' NOT NULL, KNJIZENO CHAR(1)
DEFAULT 'F' NOT NULL, STORNO CHAR(1) DEFAULT 'F' NOT NULL,
STORNOOPIS VARCHAR(50), STORNOUSER INTEGER, STORNODATE
TIMESTAMP, STORNODOK INTEGER, MODEL CHAR(2), POZIV
VARCHAR(50), TIPIFE INTEGER, KOLONAIFE INTEGER, BROJIFE
INTEGER, IDUFAVEZA INTEGER, IDAUTOZG INTEGER, IDAUTOZG2
INTEGER, IDFINZG INTEGER, IDPLACANJA INTEGER, NARUDZBA
VARCHAR(50), BROTPRAC VARCHAR(80), BRPON VARCHAR(80),
DATJCD DATE, IZNOSJCD NUMERIC(14,2), DEVIZNI CHAR(1) DEFAULT
'F' NOT NULL, DATTEC TIMESTAMP, OZNVAL VARCHAR(3), TECAJ
NUMERIC(12,6), AVOPIIS VARCHAR(30), AVIZNOS NUMERIC(14,2),
IZNRAB NUMERIC(14,2), IZNTROS NUMERIC(14,2), IZNPOR
NUMERIC(14,2), IZNBEZPOR NUMERIC(14,2), IZNSAPOR
NUMERIC(14,2), IZNOSKN NUMERIC(14,2), NAPOMENA
VARCHAR(2048), DODATNO1 VARCHAR(40), DODATNO2 VARCHAR(40),
AVRACUNI VARCHAR(50), AVSTOPA1 NUMERIC(6,1), AVOSN1
NUMERIC(14,2), AVPDV1 NUMERIC(14,2), AVSTOPA2 NUMERIC(6,1),
AVOSN2 NUMERIC(14,2), AVPDV2 NUMERIC(14,2), AVSTOPA3
NUMERIC(6,1), AVOSN3 NUMERIC(14,2), AVPDV3 NUMERIC(14,2),
FISKALNIBR VARCHAR(10), FISKALNIDATUM TIMESTAMP, FISKALNIZKI
VARCHAR(80), FISKALNIJIR VARCHAR(80), FISKALNIPARAGON
VARCHAR(20), FISKALNICTRL VARCHAR(120), ERACUN_STATUS
INTEGER, ERACUN_UPLOAD VARCHAR(64), ERACUN_PRIVITAK
VARCHAR(512), ERACUN_ID INTEGER, ERACUN_DATUM DATE,
ERACUN_XML INTEGER, ERACUN_TPP INTEGER, IRACUN_STATUS
INTEGER, IZMJENA CHAR(1) DEFAULT 'F' NOT NULL, KLJUC CHAR(1)
DEFAULT 'F' NOT NULL, SISUSER BIGINT NOT NULL, SISDATE
TIMESTAMP, CONSTRAINT VPZG_IDZG PRIMARY KEY (IDVPRZG),
CONSTRAINT VPG_FIRM FOREIGN KEY (IDFIRME) REFERENCES
FIRME(IDFIRME) ON UPDATE CASCADE, CONSTRAINT VPZG_IDPJ
FOREIGN KEY (IDPOSJED) REFERENCES POSJED(IDPOSJED) ON
UPDATE CASCADE, CONSTRAINT VPZG_IDSKL FOREIGN KEY
```

```
(IDSKLAD) REFERENCES SKLAD(IDSKLAD) ON UPDATE CASCADE,  
CONSTRAINT VPZG_USER FOREIGN KEY (SISUSER) REFERENCES  
KORISNICI(IDKOR) ON UPDATE CASCADE ); CREATE INDEX VPG_FIRM  
ON VPRZG (IDFIRME); CREATE INDEX VPZG_BRIFE ON VPRZG  
(BROJIFE); CREATE INDEX VPZG_IDPJ ON VPRZG (IDPOSJED);  
CREATE INDEX VPZG_IDSKL ON VPRZG (IDSKLAD); CREATE INDEX  
VPZG_USER ON VPRZG (SISUSER); CREATE INDEX VPZG_VK_BR ON  
VPRZG (VK DESC,BRDOK DESC);
```

Jul 8

Razmotrila strategija podudaranja s filtriranjem tipa za sigurnu povezanost zapisa

Razmotrila strategija podudaranja s filtriranjem tipa za sigurnu povezanost zapisa

Zabilježeno. Ovo mijenja `firebird_client.py` (funkcija `get_or_create_roba` treba prvo probati "goli" kod, pa tek onda prefiksirani) — uz sigurnosnu provjeru: ako goli kod postoji ali je **krivog tipa** (npr. TPR0 je zapravo artikl, TIP=2, a mi tražimo uslugu), ne smijemo ga pogrešno spojiti — moramo pasti natrag na prefiksiranu pretragu/kreiranje.

Add TIP-aware bare-code lookup before falling back to prefixed barcode matching

Add TIP-aware bare-code lookup before falling back to prefixed barcode matching

Testiram logiku grananja s lažnom konekcijom (simuliram sva tri scenarija: goli kod postoji s ispravnim tipom, goli kod postoji s krivim tipom, goli kod uopće ne postoji):

Test the two-tier lookup logic (bare code first, then prefixed) with a fake connection

Test the two-tier lookup logic (bare code first, then prefixed) with a fake connection

Oba scenarija rade ispravno.

Ažurirane funkcije u `firebird_client.py`

python

```
def find_roba(con: Connection, idfirme: int, barcode: str) -> Optional[int]:  
    """Vraca IDROBE ako artikl/usluga s danim (IDFIRME, BARCODE) vec postoji, inace  
    None."""  
    cur = con.cursor()  
    cur.execute("SELECT IDROBE FROM ROBA WHERE IDFIRME = ? AND BARCODE = ?",  
(idfirme, barcode))
```

```

row = cur.fetchone()
return row[0] if row else None

```

```

def find_roba_with_tip(con: Connection, idfirme: int, barcode: str) ->
Optional[tuple[int, int]]:
    """Vraca (IDROBE, TIP) ako zapis s danim (IDFIRME, BARCODE) postoji, inace
None."""
    cur = con.cursor()
    cur.execute("SELECT IDROBE, TIP FROM ROBA WHERE IDFIRME = ? AND BARCODE = ?",
(idfirme, barcode))
    row = cur.fetchone()
    return (row[0], row[1]) if row else None

```

```

def create_roba(con: Connection, fields: dict) -> int:
    """
    Kreira novi zapis u ROBA. 'fields' NE smije sadrzavati IDROBE - ovaj
ce ga sam dodijeliti preko GENIDROBE i vratiti novi ID.
    """
    new_id = next_id(con, "GENIDROBE")
    fields = dict(fields)
    fields["IDROBE"] = new_id
    _insert_row(con, "ROBA", fields)
    return new_id

```

```

def get_or_create_roba(
    con: Connection, idfirme: int, source: str, sifra: str, build_fields:
Callable[[], dict]
) -> int:
    """
    Trazi robu u dva koraka, pa ako ne nadje - kreira novu (s prefiksiranim
BARCODE):

```

1. NAJPRIJE trazi "goli" kod bez prefiksa (BARCODE=sifra) - radi kompatibilnosti s postojećim FB:ROBA zapisima koji su nastali PRIJE ovog alata (npr. rucno uneseni, bez A-/U- prefiksa). Ako se nade i TIP se poklapa s ocekivanjem (artikl=2 / usluga=3), koristi taj zapis. Ako TIP NE odgovara (goli kod je "zauzet" od strane drugog tipa robe - isti problem kao sukob sifri u DBF-u), preskace se i ide na korak 2.
2. Ako korak 1 ne uspije, trazi/kreira PREFIKSIRANI kod (A-sifra ili U-sifra) kao dosad - ovo je "namespace" koji ovaj alat sam kontrolira, pa tu sukoba ne moze biti.

```

Vraca IDROBE (postojeci ili novododijeljeni).
"""

```

```

tip_roba = 2 if source == "artikl" else 3

```

```

goli = find_roba_with_tip(con, idfirme, sifra)
if goli is not None:
    idrobe, postojeci_tip = goli

```



```

    if postojeci_tip == tip_roba:
        return idrobe
    # goli kod postoji, ali je krivog tipa - pada na prefiksiranu pretragu

barcode = roba_barcode(source, sifra)
existing = find_roba(con, idfirme, barcode)
if existing is not None:
    return existing
return create_roba(con, build_fields())

```

Zamijenite ovaj dio u `firebird_client.py` (funkcije `find_roba`, dodana nova `find_roba_with_tip`, `create_roba` nepromijenjena, i ažurirana `get_or_create_roba`).

Napomena za postojeći U-TPRO koji je već slučajno kreiran — ostat će "siročće" u bazi (neće se više koristiti, ali nije obrisao). Ako želite, mogu vam dati SQL da ga ručno obrišete/spojite s ispravnim TPRO zapisom, ili to ostavite za kasnije čišćenje kad završite testiranje.

Kako ću sada kompajlirati kod u izvršnu aplikaciju?

9:49 PM

Razradio strategiju kompajliranja s PyInstallerom i rješavanja putanja

Razradio strategiju kompajliranja s PyInstallerom i rješavanja putanja

Za kompajliranje Python aplikacije u samostalnu `.exe` datoteku standardni alat je **PyInstaller**. Evo koraka:

1. Instalacija

```
pip install pyinstaller
```

2. Bitna izmjena koda prije kompajliranja

Trenutno `main.py` traži `param.ini` u **trenutnom radnom direktoriju** (`load_config("param.ini")`), što radi dok pokrećete `python main.py` iz foldera projekta. Ali kad se aplikacija kompajlira u `.exe` i pokrene dvoklikom, radni direktorij nije garantirano isti kao folder gdje se `.exe` nalazi — pa treba eksplicitno odrediti putanju **pored same izvršne datoteke**.

main.py (ažurirana verzija):

```
python
```

```
"""
```

```
main.py
```

Ulazna točka aplikacije. Pokrenite s:

```
python main.py
```

ili kao kompajlirani .exe - param.ini se u oba slučaja traži u istom folderu gdje se nalazi skripta/izvršna datoteka (ne u trenutnom radnom direktoriju, koji može biti drugaciji kad se .exe pokrene dvoklikom).

```
"""
```

```
import os
import sys
```

```
def get_app_dir() -> str:
    """Vraca folder gdje se nalazi .exe (kompajlirano) ili main.py (razvoj)."""
    if getattr(sys, "frozen", False):
        return os.path.dirname(sys.executable)
    return os.path.dirname(os.path.abspath(__file__))
```

```
if __name__ == "__main__":
    os.chdir(get_app_dir()) # param.ini, dbf_reader itd. sad rade relativno na
    ovaj folder
    from gui.main_window import launch_app
    launch_app()
```

(Promjena: dodan `get_app_dir() + os.chdir(...)` prije importa GUI-ja, tako da `load_config("param.ini")` uvijek pronađe datoteku bez obzira odakle je .exe pokrenut.)

3. Kompajliranje

Iz foldera projekta (gdje se nalazi `main.py`):

```
pyinstaller --onedir --windowed --name RacunImport main.py
```

- `--onedir` (preporučeno umjesto `--onefile`) — stvara folder sa svim potrebnim datotekama umjesto jedne velike .exe, pouzdanije je za aplikacije koje pristupaju vanjskim datotekama (DBF, param.ini) i vanjskim DLL-ovima (fbclient.dll)
- `--windowed` — bez konzolnog prozora u pozadini (GUI aplikacija). **Za prvi test preporučam BEZ ove opcije** (samo `pyinstaller --onedir --name RacunImport main.py`), da vidite eventualne greške u konzoli — kad potvrdite da radi, ponovno kompajlirajte s `--windowed`.

Rezultat će biti u `dist\RacunImport\` folderu — `RacunImport.exe` plus prateće datoteke.

4. Nakon kompajliranja

Kopirajte `param.ini` u `dist\RacunImport\` folder (pored `.exe`) — PyInstaller ga automatski ne uključuje jer nije Python import, nego vanjska konfiguracija koju očekujete da ćete ručno mijenjati.

```
dist\RacunImport\  
├─ RacunImport.exe  
├─ param.ini          <- ručno kopirati ovdje  
└─ ... (ostale .dll/.pyd datoteke koje PyInstaller sam ubaci)
```

5. Napomena o `fbclient.dll`

Budući da ćete `.exe` vjerojatno pokretati na **istom računalu** gdje je Firebird server već instaliran (i gdje smo već potvrdili da veza radi), `fbclient.dll` će i dalje biti dostupan na isti način kao i dosad — PyInstaller ne mora ništa posebno raditi s njim jer ga `firebird-driver` pronalazi dinamički u vrijeme izvođenja, ne kroz Python import mehanizam.

Ako ikad budete htjeli ovaj `.exe` prenijeti na **drugo računalo** koje nema instaliran Firebird klijent, tad bi trebalo dodatno kopirati `fbclient.dll` u `dist\RacunImport\` folder i eksplicitno postaviti putanju u kodu (`driver_config.fb_client_library.value = ...`, kao što smo spomenuli u `test_konekcija.py`) — javite ako do toga dođe pa to riješimo.

Probajte prvo bez `--windowed`, pokrenite `.exe` iz `dist\RacunImport\` foldera, i javite radi li kako treba.